

Product Requirements Document

Flood Digital Twin with Outcome-Grounded Tactical Critic

Scope: Flood only. Reference scenario: Guadalupe River flash flood, Kerr County, Texas, 4 July 2025.

Status: Hackathon build (dnhacks, September 2026)

Positioning: Training and pre-deployment red-team tool. **Not** a live tactical recommender — a human commander owns every operational decision.

1. Problem Statement

Kerr County's failure on 4 July 2025 was a decision-latency failure, not a data failure. A flood watch was issued at 1:18 p.m., a flash flood warning at 1:14 a.m., and a flash flood emergency at 4:03 a.m. Gauge data, radar-estimated rainfall, a county hazard mitigation plan rating a flood event likely within a year, and an emergency operations plan prescribing reconnaissance after a watch all existed. Nobody synthesized them into an evacuation order during the 2 hours 49 minutes between warning and emergency. Nearly 200 people died.

This product closes that synthesis gap in two settings:

- 1 Training** — a trainee runs the Kerr County timeline, makes tactical decisions at fixed decision points, and is critiqued against what comparable choices cost in historical incidents.
- 2 Pre-deployment** — a draft incident action plan (IAP) is red-teamed against projected inundation and the after-action-report (AAR) corpus before a strike team leaves.

2. Goals and Non-Goals

Goals

- Fuse physics-based inundation prediction with outcome-grounded tactical critique in one session-synchronized system
- Every LLM claim traceable to a real AAR citation or a real twin feature ID — no hallucinated coordinates, no invented outcomes
- Render inundation and population/exposure data as a synchronized, timeline-scrubbable 3D scene
- Support both replay (historical Kerr County timeline) and live (real-time gauge feed) modes

Non-Goals

- Not a live incident-command recommender — outputs are training/critique artifacts, not dispatch instructions
- Not a trained hydrology model as the primary inundation method — physics comes from government hydrology models (HAND/NWM), not from an ML surrogate, except for one explicitly bounded exception (Section 6.3)
- Not a point-coordinate missing-person predictor — search-area output is a probability-weighted polygon, never a pin

3. Users and Use Cases

User	Use case
Trainee (EOC staff, incident commander in training)	Walks the Kerr County replay timeline, makes tactical calls at each decision point, receives cited critique
Pre-deployment planner / strike team lead	Submits a draft IAP; system red-teams it against projected inundation and historical AAR outcomes before deployment

User	Use case
Trainer / after-action reviewer	Exports the full decision log with citations for debrief

4. System Architecture (Reference)

Primary data path (solid arrows in the source diagram):

Data sources (hydrology, exposure layers, AAR corpus)

■

Physics + extraction (physics engine → impact extractor; AAR pipeline → RAG corpus)

■

Shared infrastructure (clock service; session state store)

■

LLM layer (critic/enrichment agent, RAG corpus, search-area tool, output validator)

■

Web UI (3D map, training mode; chatbot tab is post-MVP – see Section 6.6)

Direct bypass reads (dashed arrows in the source diagram — deliberately skip the LLM):

- **Physics engine** → **3D map**: depth/velocity raster tiles are rendered directly on the map. The LLM never touches a raster, and the map is never blocked on an LLM call to show current inundation.
- **Physics engine** → **Search-area tool**: the velocity field is read directly by the search-area tool's polygon math. The LLM does not compute probability-weighted areas itself — it only reports what the tool returns, with a citation.

These bypass paths are a deliberate design choice, not an oversight: they keep raw numerical work (rendering, geometry math) out of the LLM's hands entirely, so the LLM's only job is retrieval-grounded critique — never physics, never geometry.

Full component detail is in Section 6 (physics/rendering pipeline, including 6.6 for the LLM/RAG layer) and Section 8 (data contracts). This PRD assumes the component boundaries defined in the architecture document and adds the concrete technology choices and the rendering/physics data flow requested for implementation planning.

5. Technology Stack

Layer	Technology	Rationale
Physics engine	Python (numpy, rasterio, GDAL, xarray)	Raster math (HAND lookup, rating-curve application) is exactly numpy/rasterio's domain; no need for a compiled engine at this scale/timestep count
Geospatial intersection	Python (geopandas, shapely, rasterstats)	Standard toolkit for raster-vector zonal operations (impact extraction)
ML training/inference	PyTorch	Used narrowly, in two bounded roles — Section 6.3 and Section 6.4. Not used for the primary hydrology output.
AAR pipeline / embeddings	Python (sentence-transformers or an API embedding model), vector store (pgvector or FAISS)	Chunk-level embedding and retrieval over a small, structured corpus
Backend services	Python (FastAPI)	Clock service, impact extractor service, LLM orchestration layer, tile-serving endpoints
Geospatial database	PostGIS	Exposure layers (roads, crossings, structures, camps) with stable feature IDs, queried by both the impact extractor and the UI

Layer	Technology	Rationale
Object/tile storage	S3-compatible bucket + TiTiler/rio-tiler	Serves depth/velocity rasters as Cloud-Optimized GeoTIFF (COG) tiles on demand
Session state	Redis (pub/sub) or Postgres table with LISTEN/NOTIFY	Single source of truth for t, impact JSON, plan, LLM outputs, chat history; pushes updates to UI in real time
Frontend	TypeScript, React + Next.js	UI framework for the 3D map and training mode (MVP); the chatbot tab (Section 6.6) is post-MVP and shares the same framework when built
3D rendering	CesiumJS (preferred) or deck.gl TerrainLayer	Cesium handles quantized-mesh terrain + draped raster overlays natively; deck.gl is the faster-to-stand-up fallback if terrain tiling proves too slow to prep in hackathon time
Real-time sync	WebSocket (Socket.io or native WS)	Clock ticks and session-state deltas pushed to the frontend rather than polled

6. Rendering and Physics Data Pipeline

This is the core data flow judges and implementers need to understand: **how a raw gauge reading becomes a rendered 3D flood scene with population markers, synchronized to a scrubbable timeline.**

6.1 Ingestion

- Hydrology feeds (NOAA NWPS API, USGS Water Services) are pulled into two modes: a static July 2025 archive (replay) and a polling live feed (real-time). Both write into the same schema so downstream components are mode-agnostic.
- Terrain (USGS 3DEP 1m lidar DEM) and exposure layers (TxDOT roads, low-water crossings, NHD flowlines, OSM building footprints, camp footprints) are ingested once, at build time, into PostGIS with a stable `feature_id` assigned to every row. This ID is the only way the LLM layer is ever allowed to reference geometry — it never sees or emits a raw coordinate pair.

6.2 Physics engine (Python — the deterministic core)

For each timestep t (owned by the clock service):

- Look up per-reach streamflow (from the hydrology feed) for t .
- Convert flow $\rightarrow$ stage via the basin's synthetic rating curve.
- Compute cell-wise depth: $\text{stage} - \text{HAND}[\text{cell}]$, using the precomputed HAND raster for the relevant HUC8.
- Compute a velocity proxy from local slope, depth, and a roughness coefficient (Manning-style approximation) — labeled explicitly as a proxy in all outputs, since HAND provides depth only, not velocity.
- Write depth and velocity as a GeoTIFF pair per timestep.

This engine is **headless** — no rendering happens here. That separation is what allows batch pre-rendering for replay mode and repeated tool calls from the LLM layer without a GPU renderer in the critical path.

6.3 Bounded learned component #1 — residual correction (PyTorch, stretch goal)

The one place a trained model is allowed to touch the primary inundation output, and only as a correction layer, never as the source:

- Training data:** historical HAND-based inundation outputs for the same basin, paired with observed high-water marks and satellite-derived inundation extents from past floods.

- **Model:** a lightweight regression model (a small feed-forward network or a shallow CNN over local raster patches) predicting a per-cell depth correction, in PyTorch.
- **Inference:** applied as a fast forward pass on the physics engine's raw depth raster before it's handed to the impact extractor — must run within the per-timestep render budget (see Section 6.6).
- **Why bounded this way:** it corrects a known, documented failure mode (HAND underprediction on smaller streams) without replacing the physics-based method as the system's primary claim. It is optional; the system must run and be demoable with this component entirely disabled.

6.4 Bounded learned component #2 — historical-decision outcome scorer (PyTorch, stretch goal — optional, does not block demo)

This is new relative to the original architecture document's "nothing is trained" framing for tactics, and is worth stating explicitly as a deliberate scope decision. **Priority: low.** Build this only after Milestones 1–4 (Section 9) are complete and stable. It should never block, delay, or replace the core citation-grounded critique path.

Default shipped behavior: the nearest-neighbor similarity fallback described below (option 2) is the recommended default for the demo, since it requires zero training data, is inherently citation-traceable, and avoids the credibility risk of presenting an undertrained model's output as a prediction. The trained-model path (option 1) is an explicit stretch add-on, attempted only with time to spare and always shown alongside — never instead of — the similarity-based signal.

- **Purpose:** supplement the LLM critic's qualitative, citation-grounded critique with a quantitative signal — given a proposed tactic at a given decision point, how did similar tactics perform in the historical record?
- **Training data:** the same structured AAR corpus used for RAG (hazard, phase, tactic, resources, outcome, lesson, citation), reshaped into (situation features, tactic taken, outcome severity) tuples.
- **Model:** given the very small number of usable historical incidents (a handful of Texas flash-flood AARs), a deep model will overfit badly — this is the same data-scarcity problem documented in the flood/hurricane evacuation-modeling research this project builds on. Recommended approach, in order of preference:
 - 1 **Preferred:** a shallow model — logistic/ordinal regression or a small feed-forward network in PyTorch over hand-engineered features (rainfall rate, warning lead time, tactic category, resource count) — trained with heavy regularization and leave-one-incident-out validation.
 - 2 **Fallback if even that overfits:** drop the trained model entirely and use nearest-neighbor retrieval similarity (already available via the RAG embeddings) as the quantitative score instead — cosine similarity to the closest historical case, with the outcome of that case surfaced directly. This is defensible with zero training data and should be the default if time is short.
- **Output:** a bounded score (e.g., 0–1 severity/risk estimate) attached to the LLM critic's response, never replacing the cited natural-language critique — the model scores, the LLM (grounded in retrieval) explains.
- **Guardrail:** this scorer's output must never be presented without the citation-grounded critique alongside it. A number with no citation is exactly the kind of ungrounded claim Design Principle 3 (Section 7) exists to prevent.

6.5 Impact extractor

Intersects the (corrected, if enabled) depth and velocity rasters with the PostGIS exposure layers using rasterstats/geopandas zonal operations. Emits impact JSON per timestep: impassable crossings now and at t+30/60/120 minutes, threatened structures with depth, camp egress routes and the time each goes under, reach-level stage and rate of rise. This JSON — never the raster — is what the LLM layer is allowed to read.

6.6 LLM/RAG Layer

This section gives the LLM/RAG layer its own dedicated specification, matching the source architecture diagram's "LLM layer" box — it's the reasoning layer sitting between the impact extractor and the Web UI, and it never touches a raw raster or a raw coordinate.

Two roles, one shared session state:

- **Enrichment agent** — runs each timestep (or on-demand), consuming the current t , the impact JSON, and RAG retrieval results, and produces the overlay JSON that drives the 3D map's markers and polygons. This is core MVP — the map has no population/hazard annotations without it.
- **Chat agent (post-MVP / stretch goal)**. Powers an open-ended conversational chatbot tab over the same session state and tools. **This is not required for the MVP demo.** Training mode's decision-point critique already delivers the core citation-grounded-critique experience using the same underlying critic logic, without needing a general-purpose chat interface. Build the chatbot only after Milestones 1–5 are complete and stable, and treat it as the lowest-priority item in the entire system — it adds conversational convenience, not new capability.

Tools available to both roles:

- Impact extractor query — by feature ID or area, at a given t
- RAG corpus retrieval — by hazard type, phase, and semantic similarity to the current impact JSON and/or the trainee's proposed plan
- Search-area tool (below)

Critic behavior: given a trainee's proposed tactics at a decision point, retrieve the closest historical incidents from the AAR corpus, compare the proposal against what worked and failed in those incidents, and return specific objections and alternatives — every objection citing a chunk ID or a feature ID, never a bare assertion.

Search-area tool: deliberately reframed from "predicted missing-person coordinates" to search-area **prioritization**. Inputs: last-known positions (scripted call data in the Kerr County replay), the physics engine's velocity field (read directly — the bypass connection noted in Section 4), and the depth raster. Output: probability-weighted downstream polygons with explicit uncertainty — never a single point. This framing matters operationally: land-search behavior models don't transfer to swiftwater conditions, and a tool that returns one coordinate for one person is the exact output a judge or reviewer with SAR experience will attack first.

Output validator: a post-processing step run on every LLM output — enrichment or chat — that rejects any response referencing a chunk ID or feature ID not present in the corpus or the current impact JSON. Rejected outputs are regenerated, not passed through silently. This validator is what actually enforces Design Principle 3 (Section 7) — without it, the citation requirement is just a prompt instruction, not a guarantee.

6.7 Tiling and rendering

- 1 **Tiling:** depth/velocity GeoTIFFs are converted to Cloud-Optimized GeoTIFFs and served as XYZ/TMS tiles via `TiTiler/rio-tiler`. For replay mode, tiles for the full Kerr County timeline are pre-rendered in batch ahead of the demo, removing render latency from the critical path entirely. For live mode, tiles are generated on demand per incoming timestep, budgeted to complete within the polling interval of the hydrology feed (15-minute USGS resolution gives ample headroom).
- 2 **Terrain base layer:** the 1m lidar DEM is converted to a Cesium-compatible quantized-mesh tileset (via `cesium-terrain-builder` or equivalent) once, at build time.
- 3 **Frontend render (TypeScript/CesiumJS):** the map loads the static terrain tileset, drapes the current-timestep depth tile as a color-ramped overlay, and renders the LLM enrichment agent's overlay JSON (markers, polygons — feature-ID-referenced only) as vector primitives on top.
- 4 **Timeline sync:** the clock service pushes t over WebSocket; the frontend's timeline scrubber both reflects and can drive t (in replay mode), triggering a tile-layer swap and a session-state query for the impact JSON and LLM outputs at the new t .
- 5 **Training mode** reads the same session-state store as the map, so a training-mode critique and the map overlay can never contradict each other — they're rendered from one shared state object, not two independent LLM calls. The chatbot tab (6.6), once built, reads the same store for the same reason.

6.8 Known limitations (state these in the demo, not just this document)

- HAND underpredicts on fourth-order and smaller streams and is sensitive to the roughness coefficient; the Guadalupe main stem is large enough for the method, but the tributary creeks that hit the camps are borderline.

- The velocity field is an explicit proxy, not a direct measurement.
- Lead time for this flood type is minutes to a few hours, driven by observed upstream flow — not a forecast issued the afternoon before.
- The historical-decision scorer (6.4) is trained/validated on a very small number of incidents; treat its output as a rough prior, not a calibrated probability.

7. Design Principles (carried forward, one amended)

- 1 Physics does physics, the LLM does tactics, and — new — a narrowly bounded PyTorch model may score tactics numerically, but never explains them without a citation-grounded LLM statement alongside it.
- 2 The primary inundation output is grounded in government hydrology models, not trained end-to-end. The two exceptions (6.3, 6.4) are explicitly bounded, optional, and clearly labeled as learned components in every output they touch.
- 3 Every LLM claim cites either an AAR chunk ID or a twin feature ID. Outputs referencing IDs absent from the corpus or current impact JSON are rejected by the output validator and regenerated.
- 4 One clock. Every component reads simulation time from a single service.
- 5 Contracts before code. Data contracts (Section 8) are agreed and stubbed with canned Kerr County data before any layer is optimized.

8. Data Contracts

- 1 **Engine** → **Extractor**: raster format (GeoTIFF, CRS, resolution), depth and velocity bands, timestep t , reach IDs.
- 2 **Extractor** → **LLM**: impact JSON schema — feature IDs, projected horizons ($t+30/60/120$), stage/rate-of-rise metrics.
- 3 **LLM** → **UI**: overlay JSON schema — feature-ID references only, type (marker/polygon/timed event), severity, citation IDs, applicable t .
- 4 **Session state schema**: t , mode (live/replay), impact JSON, trainee plan, LLM outputs (including the 6.4 scorer output where enabled), chat history, decision-point status.

Each contract should be stubbed with canned Kerr County data and rendered end-to-end in the UI before any individual layer is optimized — this validates the pipeline shape before investing in physics or model accuracy.

9. Milestones and Team Allocation

Milestone	Deliverable
Hour 0–1	Contracts agreed and stubbed (Section 8)
Milestone 1	Static terrain tileset + PostGIS exposure layers loaded; empty map renders in CesiumJS
Milestone 2	Physics engine produces depth/velocity rasters for the Kerr County replay timeline; tiles render on the map, timeline scrubber works
Milestone 3	Impact extractor + AAR pipeline live; LLM critic responds with cited critique at a single decision point
Milestone 4	All three Kerr County decision points wired into training mode; search-area tool returns polygons at the 4:03 a.m. point
Milestone 5 (stretch goal — optional, does not block demo)	PyTorch residual correction model (6.3) and/or historical-decision scorer (6.4) integrated, built only after Milestones 1–4 are complete, presented as an early proof-of-concept with stated data limitations, never as a calibrated prediction

Milestone	Deliverable
Milestone 6 (post-MVP — lowest priority in the system)	Chatbot tab (Section 6.6) built only if Milestones 1–5 are complete and time remains; not required for the demo, since training mode already delivers the core critique experience

Suggested three-person split (from the source architecture, unchanged): one on physics engine + impact extractor; one on AAR pipeline, RAG corpus, and search-area tool; one on clock service, session state, LLM layer, and web UI. If a fourth person is available, they should own the PyTorch components (6.3/6.4) exclusively, since both are explicitly optional and should not block the core path if they run out of time.

10. Cut List (in order, if time runs short)

- 1 The chatbot tab (Section 6.6) is already out of MVP scope — there's nothing to cut because it was never scheduled before Milestone 6. Do not start it until everything below is done.
- 2 Drop live mode — replay only.
- 3 Drop the historical-decision scorer (6.4) — fall back to nearest-neighbor similarity display with no trained model, or omit entirely.
- 4 Drop the residual correction model (6.3) — present raw HAND-based depth with limitations stated.
- 5 Drop the 3D viewer (CesiumJS) for a 2D depth map (e.g., `deck.gl` or even a static Leaflet choropleth).
- 6 Reduce the AAR corpus to the Kerr County documents alone.

11. Risks and Assumptions

- **Data scarcity for any trained component.** Both PyTorch components (6.3, 6.4) are trained on very small historical samples. This is the same limitation documented in prior evacuation-behavior research this project draws on (e.g., a handful of usable hurricane/wildfire events nationally) — the mitigation is the same: prefer shallow, regularized models or pure similarity-based retrieval over deep learning, and validate with leave-one-incident-out testing, not random splits.
- **HAND/velocity-proxy accuracy on small tributaries** is a stated, not hidden, limitation — the demo should say this out loud rather than let a judge discover it.
- **Citation integrity** depends entirely on the output validator (Section 7, principle 3) actually rejecting and regenerating non-compliant outputs — this should be tested adversarially before the demo, not assumed to work.
- **Real-time tile generation latency** in live mode is unbounded if not budgeted against the hydrology feed's polling interval — pre-rendering for replay mode sidesteps this risk entirely, which is one more reason replay should be the default demo path.

12. Human/Population Data Sourcing (Reference)

The core system uses scripted call data for last-known positions (Section 6.6) and does not depend on real population/mobility data for the MVP. This section is a reference for a future extension — enriching camp/structure occupancy with real population or behavioral data — not a hackathon dependency.

Parameter	Source(s)	Access model
Anonymized smartphone/GPS ping data	Cuebiq (Social Impact / Data for Good, now continued via Spectus.ai); SafeGraph (via the Dewey platform); Veraset	Proposal-vetted research application (Cuebiq); university-subscription academic access (Dewey/SafeGraph); commercial license only (Veraset)

Parameter	Source(s)	Access model
Census block group data	U.S. Census Bureau ACS 5-year estimates; TIGER/Line shapefiles; CDC/ATSDR Social Vulnerability Index	Free, public
Post-disaster surveys	ICPSR archives; Natural Hazards Center Quick Response/CONVERGE program; FEMA AAR-embedded survey data	Free where archived; primary data collection otherwise
Web search behavior	Google Trends (public UI/API); Google Trends via BigQuery	Free, public; limited to metro/state-level granularity, not block-group
Sociodemographic characteristics	U.S. Census ACS; HUD data; IRS Statistics of Income; USDA Rural-Urban Continuum Codes	Free, public
Prior mobility patterns	Same providers as GPS ping data, pulled as a historical archive window rather than a live feed	Same access model as row 1

Feasibility note for this hackathon: neither Cuebiq's Social Impact program nor Dewey/SafeGraph access can be obtained same-day — Cuebiq is a vetted research-proposal process with no published turnaround guarantee, and Dewey requires both an existing institutional subscription and a separate admin approval step that multiple universities document as taking up to 48 hours. For this build, use the free/public sources (Census/SVI, Google Trends) directly, and represent any GPS-mobility-dependent behavior with the clearly-labeled synthetic generator already specified elsewhere in this document, rather than blocking on a data-access approval that won't clear in time.